Phase 1, Part 2 Report

Task 1: Benchmarking

A reusable benchmarking framework was built using std::chrono::high_resolution_clock. Each function was timed over multiple iterations, and the mean and standard deviation of execution times were computed. Tests were run across matrix sizes of 64, 256, 512, and 1024 (square matrices) for matrix-matrix multiplication, and up to 4096 for matrix-vector multiplication. Baseline benchmarks were compiled with -O0 (no optimization) to reflect raw algorithm performance.

Matrix-Vector Multiplication Results (-O0)

Function	Size	Mean (ms)	Std Dev (ms)	Runs
Row-Major	64x64	0.009	0.006	1000
Col-Major	64x64	0.008	0.003	1000
Row-Major	256x256	0.182	0.037	100
Col-Major	256x256	0.146	0.077	100
Row-Major	1024x1024	3.185	0.710	50
Col-Major	1024x1024	1.783	0.658	50
Row-Major	4096x4096	46.525	3.850	10
Col-Major	4096x4096	29.019	1.755	10

Column-major multiplication was consistently faster than row-major across all sizes, with the gap widening at larger sizes. At 4096x4096, column-major was approximately 1.6x faster. This is discussed further in the cache locality analysis.

Matrix-Matrix Multiplication Results (-O0)

Function	Size	Mean (ms)	Std Dev (ms)	Runs
Naive	64x64	0.614	0.259	100
Transposed-B	64x64	0.623	0.261	100
Tiled	64x64	0.494	0.146	100
Naive	256x256	40.626	1.684	10
Transposed-B	256x256	39.209	2.045	10
Tiled	256x256	32.202	2.868	10
Naive	512x512	325.934	4.073	5
Transposed-B	512x512	317.792	5.803	5
Tiled	512x512	258.969	5.715	5
Naive	1024x1024	4323.640	10.739	3

Transposed-B	1024x1024	2540.729	12.331	3
Tiled	1024x1024	2060.334	19.033	3

At 1024x1024 without compiler optimization, the transposed-B approach was approximately 1.7x faster than naive, and the tiled implementation was 2.1x faster than naive. The tiled version outperformed both baselines across all sizes. The performance differences are less dramatic at -O0 than they would be with optimization enabled, since the compiler cannot exploit the cache-friendly access patterns through vectorization.

Task 2: Cache Locality Analysis

To isolate cache effects, a stride-access benchmark was designed. A 64MB array of doubles was allocated, and every Nth element was summed for strides of 1 through 128. The time per element accessed was computed to normalize for the different amounts of work.

Cache Stride Analysis (-O0)

Stride	Mean (ms)	Elements Accessed	ns/element
1	23.242	8,388,608	2.77
2	11.936	4,194,304	2.85
4	6.680	2,097,152	3.19
8	3.719	1,048,576	3.55
16	3.185	524,288	6.08
32	1.872	262,144	7.14
64	0.995	131,072	7.59
128	0.460	65,536	7.02

The cost per element increased from 2.77 ns at stride 1 to over 7 ns at stride 64, a roughly 2.7x increase. A notable jump occurs between stride 8 and stride 16, which corresponds to exceeding the CPU cache line boundary: a cache line is 64 bytes, which holds exactly 8 doubles. At stride 16 and beyond, every access is guaranteed to miss the prefetched cache line data, and the cost per element roughly doubles.

This explains the performance difference between naive and transposed-B matrix multiplication: the naive version performs strided access to B, while the transposed version converts this into sequential access, improving cache utilization.

For matrix-vector multiplication, column-major storage outperformed row-major. In the column-major implementation, the outer loop iterates over columns, streaming through the matrix and vector sequentially. The row-major version performs a dot product per row, repeatedly accessing the vector and striding through the matrix, which leads to more cache misses at larger sizes.

Task 3: Memory Alignment

Memory alignment was tested by comparing standard heap allocation (using new) against 64-byte aligned allocation (using _aligned_malloc on Windows). The same kernel functions were benchmarked with identical input data.

Aligned vs Unaligned Results (-O0)

Function	Size	Unaligned (ms)	Aligned (ms)	Difference
Naive	256x256	38.878	40.147	+3.3%
Trans-B	256x256	38.578	38.629	+0.1%
Naive	512x512	327.209	325.169	-0.6%
Trans-B	512x512	318.299	312.675	-1.8%
Naive	1024x1024	4246.153	4188.208	-1.4%
Trans-B	1024x1024	2544.004	2760.104	+8.5%

Memory alignment did not produce a consistent or significant performance improvement. For naive multiplication, aligned memory was marginally slower in some cases, while transposed-B showed small improvements of 3-6% with alignment. The differences are within noise for the number of iterations used.

This is expected on modern x86 hardware: the CPU handles unaligned access to doubles efficiently, and the standard allocator (new) already returns memory aligned to at least 8 or 16 bytes, which is sufficient for double arithmetic. The performance bottleneck in these operations is cache access patterns, not memory alignment. Alignment would matter more for vectorized operations that require specific alignment boundaries.

Task 4: Inlining and Compiler Optimization

The same benchmarks were compiled with -O0 and -O3.

Compiler Optimization Comparison

Function	-O0 (ms)	-O3 (ms)	Speedup
MV Row-Major (4096)	46.525	10.038	4.6x
MV Col-Major (4096)	29.019	8.177	3.5x
MM Naive (1024)	4323.640	3443.423	1.3x
MM Trans-B (1024)	2540.729	498.014	5.1x
MM Tiled (1024)	2060.334	559.447	3.7x

Compiler optimizations had the largest impact on the transposed-B implementation, which saw a 5.1x speedup from -O0 to -O3. This is because the sequential memory access pattern of transposed-B is ideal for SIMD vectorization and loop unrolling, which -O3 enables. The naive implementation benefited much less (only 1.3x) because its strided access pattern limits what the compiler can optimize. The tiled implementation saw a 3.7x speedup, reflecting its cache-friendly but more complex loop structure.

This demonstrates that compiler optimizations amplify cache-friendly code. Writing cache-aware algorithms and letting the compiler optimize them yields multiplicative performance gains, whereas poor access patterns limit optimization potential regardless of compiler settings.

No explicit inline keyword was used in the kernel functions, as they are not small helper functions. The compiler's own inlining decisions at -O3 are generally superior to manual annotation for functions of this size.

Task 5: Profiling

Profiling was performed using gprof on WSL. The code was compiled with -O0 -pg -g and run with the --mm-only flag to focus on matrix-matrix multiplication.

gprof Flat Profile Results

Function	% Time	Self (s)	Calls	s/call
multiply_mm_naive	58.66%	13.52	118	0.11
multiply_mm_transposed_b	41.04%	9.46	118	0.08
benchmark_mm	0.17%	0.04	4	0.01
fill_random	0.13%	0.03	8	0.00

The two kernel functions account for 99.7% of total execution time. The naive implementation consumed 58.66% of runtime (13.52s) compared to 41.04% for transposed-B (9.46s), reflecting the cache performance difference. All other functions, including the benchmarking infrastructure, timing, memory allocation, and string formatting, contributed negligible overhead.

The profiling confirms that optimization efforts should focus exclusively on the multiply kernels, as no other part of the program is a meaningful bottleneck.

```
Flat profile:

Each sample counts as 0.01 seconds.
  %   cumulative   self              self     total
 time   seconds   seconds    calls  s/call   s/call  name
 58.66     13.52    13.52      118    0.11     0.11  multiply_mm_naive(double const*, int, int, double const*, int, int, double*)
 41.04     22.98     9.46      118    0.08     0.08  multiply_mm_transposed_b(double const*, int, int, double const*, int, int, do
  0.17     23.02     0.04        4    0.01     5.76  benchmark_mm(int, int)
  0.13     23.05     0.03        8    0.00     0.00  fill_random(double*, int)
  0.00     23.05     0.00      976    0.00     0.00  __gnu_cxx::__normal_iterator<double*, std::vector<double, std::allocator<doub
  0.00     23.05     0.00      708    0.00     0.00  std::chrono::duration<long, std::ratio<1l, 1000000000l> >::count() const
  0.00     23.05     0.00      488    0.00     0.00  bool __gnu_cxx::operator!=<double*, std::vector<double, std::allocator<double
  0.00     23.05     0.00      472    0.00     0.00  __gnu_cxx::__normal_iterator<double*, std::vector<double, std::allocator<doub
  0.00     23.05     0.00      472    0.00     0.00  __gnu_cxx::__normal_iterator<double*, std::vector<double, std::allocator<doub
  0.00     23.05     0.00      472    0.00     0.00  std::chrono::time_point<std::chrono::_V2::system_clock, std::chrono::duration
```

gprof flat profile output from terminal

Task 6: Optimization via Cache Tiling

Based on the cache locality analysis and profiling results, a tiled (blocked) matrix-matrix multiplication was implemented as the primary optimization. The key insight is that the naive implementation repeatedly loads and evicts matrix B from cache for each row of A, resulting in excessive cache misses at large matrix sizes.

Reasoning Behind Using Tiling

In the naive implementation, each row of A is loaded into cache and used to compute one row of the result. During this computation, all of B is streamed through cache sequentially. By the time the next row of A begins, the early portions of B have been evicted, requiring a full reload from main memory. For a 1024×1024 matrix, this means B is effectively reloaded 1024 times.

Tiling restructures the loop nest to operate on T×T subblocks, keeping the working set small enough to remain in L1 cache throughout the inner computation. With a tile size of 32, each block occupies 8 KB, and three concurrent blocks (from A, B, and the result) total 24 KB — well within L1 capacity. The inner loop order (i-k-j) hoists the A[i][k] access outside the innermost loop, allowing sequential access to both B and the result within each tile. This reduces the number of times B is reloaded from 1024 (once per row of A) to 32 (once per block of A rows), cutting memory traffic by a factor of the tile size.

Tiled vs Baseline Results (-O0)

Size	Naive (ms)	Transposed-B (ms)	Tiled (ms)	Tiled Speedup vs Naive
64x64	0.614	0.623	0.494	1.2x
256x256	40.626	39.209	32.202	1.3x
512x512	325.934	317.792	258.969	1.3x
1024x1024	4323.640	2540.729	2060.334	2.1x

The tiled implementation achieved a 2.1x speedup over naive at 1024x1024 without compiler optimization, and outperformed transposed-B at all sizes. The speedup grows with matrix size as expected, since the cache reuse benefit becomes more pronounced when matrices exceed cache capacity. At 64x64, where all data fits in L1 cache, the advantage is minimal.

When combined with -O3 compiler optimization (see Task 4), the tiled version achieves a 3.7x speedup over its own -O0 performance, demonstrating that tiling and compiler optimization provide complementary benefits.

The tiling optimization validates the cache locality analysis: the dominant performance bottleneck in large matrix multiplication is cache misses from repeated reloading of matrix data, and restructuring the computation to improve cache reuse yields order-of-magnitude improvements.

Appendix: Output Screenshots

‘make test’ output:

```
PS C:\Users\chois\FINMATH\Advanced-Computing\FINM-32700-Phases> make test
./tests.exe
Running kernel tests...
PASS: multiply_mv_row_major
PASS: multiply_mv_col_major
PASS: multiply_mm_naive
PASS: multiply_mm_transposed_b
PASS: multiply_mm_tiled
```

‘make noopt’ output:

```
=== Matrix-Vector Benchmarks ===
MV Row-Major    (64x64)              mean:      0.009 ms   stddev:      0.006 ms   (1000 runs)
MV Col-Major    (64x64)              mean:      0.008 ms   stddev:      0.003 ms   (1000 runs)
MV Row-Major    (256x256)            mean:      0.182 ms   stddev:      0.037 ms   (100 runs)
MV Col-Major    (256x256)            mean:      0.146 ms   stddev:      0.077 ms   (100 runs)
MV Row-Major    (1024x1024)          mean:      3.185 ms   stddev:      0.710 ms   (50 runs)
MV Col-Major    (1024x1024)          mean:      1.783 ms   stddev:      0.658 ms   (50 runs)
MV Row-Major    (4096x4096)          mean:     46.525 ms   stddev:      3.850 ms   (10 runs)
MV Col-Major    (4096x4096)          mean:     29.019 ms   stddev:      1.755 ms   (10 runs)

=== Matrix-Matrix Benchmarks ===
MM Naive        (64x64)              mean:      0.614 ms   stddev:      0.259 ms   (100 runs)
MM Transposed-B (64x64)              mean:      0.623 ms   stddev:      0.261 ms   (100 runs)
MM Tiled        (64x64)              mean:      0.494 ms   stddev:      0.146 ms   (100 runs)
MM Naive        (256x256)            mean:     40.626 ms   stddev:      1.684 ms   (10 runs)
MM Transposed-B (256x256)            mean:     39.209 ms   stddev:      2.045 ms   (10 runs)
MM Tiled        (256x256)            mean:     32.202 ms   stddev:      2.868 ms   (10 runs)
MM Naive        (512x512)            mean:    325.934 ms   stddev:      4.073 ms   (5 runs)
MM Transposed-B (512x512)            mean:    317.792 ms   stddev:      5.803 ms   (5 runs)
MM Tiled        (512x512)            mean:    258.969 ms   stddev:      5.715 ms   (5 runs)
MM Naive        (1024x1024)          mean:   4323.640 ms   stddev:     10.739 ms   (3 runs)
MM Transposed-B (1024x1024)          mean:   2540.729 ms   stddev:     12.331 ms   (3 runs)
MM Tiled        (1024x1024)          mean:   2060.334 ms   stddev:     19.033 ms   (3 runs)

=== Aligned vs Unaligned Memory ===
MM Naive Unaligned  (256x256)        mean:     38.878 ms   stddev:      1.870 ms   (10 runs)
MM Naive Aligned    (256x256)        mean:     40.147 ms   stddev:      1.125 ms   (10 runs)
MM Trans-B Unaligned(256x256)        mean:     38.578 ms   stddev:      1.359 ms   (10 runs)
MM Trans-B Aligned  (256x256)        mean:     38.629 ms   stddev:      3.120 ms   (10 runs)
MM Naive Unaligned  (512x512)        mean:    327.209 ms   stddev:      5.129 ms   (5 runs)
MM Naive Aligned    (512x512)        mean:    325.169 ms   stddev:      6.885 ms   (5 runs)
MM Trans-B Unaligned(512x512)        mean:    318.299 ms   stddev:     14.485 ms   (5 runs)
MM Trans-B Aligned  (512x512)        mean:    312.675 ms   stddev:      4.646 ms   (5 runs)
MM Naive Unaligned  (1024x1024)      mean:   4246.153 ms   stddev:      7.679 ms   (3 runs)
MM Naive Aligned    (1024x1024)      mean:   4188.208 ms   stddev:     69.377 ms   (3 runs)
MM Trans-B Unaligned(1024x1024)      mean:   2544.004 ms   stddev:     10.837 ms   (3 runs)
MM Trans-B Aligned  (1024x1024)      mean:   2760.104 ms   stddev:    116.137 ms   (3 runs)

=== Cache Stride Analysis ===
(Summing every Nth element of a 64MB array)
Stride 1    mean:     23.242 ms   elements:    8388608   ns/element:     2.77
Stride 2    mean:     11.936 ms   elements:    4194304   ns/element:     2.85
Stride 4    mean:      6.680 ms   elements:    2097152   ns/element:     3.19
Stride 8    mean:      3.719 ms   elements:    1048576   ns/element:     3.55
Stride 16   mean:      3.185 ms   elements:     524288   ns/element:     6.08
Stride 32   mean:      1.872 ms   elements:     262144   ns/element:     7.14
Stride 64   mean:      0.995 ms   elements:     131072   ns/element:     7.59
Stride 128  mean:      0.460 ms   elements:      65536   ns/element:     7.02
```

‘make run’ output:

```
=== Matrix-Vector Benchmarks ===
MV Row-Major    (64x64)          mean:      0.001 ms   stddev:      0.001 ms   (1000 runs)
MV Col-Major    (64x64)          mean:      0.001 ms   stddev:      0.000 ms   (1000 runs)
MV Row-Major    (256x256)        mean:      0.022 ms   stddev:      0.002 ms   (100 runs)
MV Col-Major    (256x256)        mean:      0.018 ms   stddev:      0.006 ms   (100 runs)
MV Row-Major    (1024x1024)      mean:      0.608 ms   stddev:      0.294 ms   (50 runs)
MV Col-Major    (1024x1024)      mean:      0.299 ms   stddev:      0.115 ms   (50 runs)
MV Row-Major    (4096x4096)      mean:      9.390 ms   stddev:      0.142 ms   (10 runs)
MV Col-Major    (4096x4096)      mean:      8.190 ms   stddev:      0.819 ms   (10 runs)

=== Matrix-Matrix Benchmarks ===
MM Naive        (64x64)          mean:      0.077 ms   stddev:      0.009 ms   (100 runs)
MM Transposed-B (64x64)          mean:      0.110 ms   stddev:      0.051 ms   (100 runs)
MM Tiled        (64x64)          mean:      0.055 ms   stddev:      0.021 ms   (100 runs)
MM Naive        (256x256)        mean:     13.375 ms   stddev:      1.792 ms   (10 runs)
MM Transposed-B (256x256)        mean:      5.416 ms   stddev:      0.876 ms   (10 runs)
MM Tiled        (256x256)        mean:      3.147 ms   stddev:      1.028 ms   (10 runs)
MM Naive        (512x512)        mean:    136.404 ms   stddev:      7.290 ms   (5 runs)
MM Transposed-B (512x512)        mean:     60.178 ms   stddev:      3.074 ms   (5 runs)
MM Tiled        (512x512)        mean:     23.418 ms   stddev:      1.425 ms   (5 runs)
MM Naive        (1024x1024)      mean:   3245.561 ms   stddev:     80.625 ms   (3 runs)
MM Transposed-B (1024x1024)      mean:    483.341 ms   stddev:     24.034 ms   (3 runs)
MM Tiled        (1024x1024)      mean:    197.069 ms   stddev:      3.501 ms   (3 runs)

=== Aligned vs Unaligned Memory ===
MM Naive Unaligned  (256x256)    mean:     12.383 ms   stddev:      1.590 ms   (10 runs)
MM Naive Aligned    (256x256)    mean:     12.759 ms   stddev:      1.682 ms   (10 runs)
MM Trans-B Unaligned(256x256)    mean:      5.581 ms   stddev:      1.295 ms   (10 runs)
MM Trans-B Aligned  (256x256)    mean:      5.680 ms   stddev:      1.091 ms   (10 runs)
MM Naive Unaligned  (512x512)    mean:    114.277 ms   stddev:      6.124 ms   (5 runs)
MM Naive Aligned    (512x512)    mean:    116.716 ms   stddev:      5.628 ms   (5 runs)
MM Trans-B Unaligned(512x512)    mean:     51.986 ms   stddev:      2.644 ms   (5 runs)
MM Trans-B Aligned  (512x512)    mean:     50.989 ms   stddev:      3.370 ms   (5 runs)
MM Naive Unaligned  (1024x1024)  mean:   3297.222 ms   stddev:     19.762 ms   (3 runs)
MM Naive Aligned    (1024x1024)  mean:   3560.385 ms   stddev:    300.627 ms   (3 runs)
MM Trans-B Unaligned(1024x1024)  mean:    918.010 ms   stddev:    208.400 ms   (3 runs)
MM Trans-B Aligned  (1024x1024)  mean:   1192.175 ms   stddev:     43.871 ms   (3 runs)

=== Cache Stride Analysis ===
(Summing every Nth element of a 64MB array)
Stride 1    mean:      9.684 ms   elements:    8388608   ns/element:     1.15
Stride 2    mean:      5.799 ms   elements:    4194304   ns/element:     1.38
Stride 4    mean:      4.833 ms   elements:    2097152   ns/element:     2.30
Stride 8    mean:      4.264 ms   elements:    1048576   ns/element:     4.07
Stride 16   mean:      2.798 ms   elements:     524288   ns/element:     5.34
Stride 32   mean:      2.224 ms   elements:     262144   ns/element:     8.48
Stride 64   mean:      1.323 ms   elements:     131072   ns/element:    10.09
Stride 128  mean:      0.720 ms   elements:      65536   ns/element:    10.98
```